

Contents

03-implementation-and-operations-handoff	1
Implementation and operations handoff	1
Choose and inspect a bot	1
Configure persistent and authorized bots	2
Moderation boundaries	3
Offline validation and live operation	4
Implementation references	4

03-implementation-and-operations-handoff

Implementation and operations handoff

The repository contains thirteen Slack examples. Each uses `require("slack")` and Slack-native services; no Discord context or compatibility namespace is required. The ports preserve supported workflows, while command argument entry, context menus and permission models follow Slack's capabilities.

Choose and inspect a bot

```
go run ./cmd/slack-bot bots list
go run ./cmd/slack-bot bots inspect knowledge-base
go run ./cmd/slack-bot bots manifest knowledge-base
```

Inspection lists commands, events, actions, views, shortcuts, options, local verbs and declared configuration. It does not open databases or contact Slack. The manifest is the complete app configuration for this bot, including required scopes. Some examples share command names, notably `/kb-search`. Use separate apps for simultaneously running examples, or intentionally switch the single app being developed. Starting a different script synchronizes that app's manifest. If scopes change, the CLI asks for reinstallation before connecting.

Bot	Entry points and workflows
announcements	/announce-preview title: Block Kit announcement preview.
ping	/golem-ping, /golem-echo, /golem-feedback, /golem-search, /golem-announce: controls, private follow-up, form, suggestions and generated report file. Text trigger !pingjs.
hater	/hate, /roast [@user], /compliment: humor controls, apology form and verdict. Text triggers !hate, !roastme.
interaction-types	/hello, /echo, /fun roll [sides], /fun coin, /show-avatar; Quote Message shortcut.
unified-demo	/unified-ping, /unified-status; local bots invoke unified-demo status and run metadata verbs. API key values are redacted.

Bot	Entry points and workflows
poker	/poker-help, deal, draw, score, reset, rank, action with the /poker- prefix; rank/advice modals and isolated rounds.
support	/support-ticket topic, /support-status, /support-fetch-thread parent-ts, /support-start-thread topic, /support-join-channel channel-id, /support-leave-channel.
custom-kb	/kb-add, /kb-link url \ title \ summary \ tags, /kb-search, /kb-list; persistent links, selection and refresh.
knowledge-base	/teach, /remember, /ask, /kb-search, /article, /kb-article, /review, /kb-review, /recent, /kb-recent, /kb-verify, /kb-stale, /kb-reject; capture, curation, editing, pagination, source and export.
show-space	/upcoming, /announce, /add-show, /show id, /cancel-show id, /archive-show id, /past-shows, /unpin-old, /archive-expired; diagnostic commands /debug, /debug-groups, /debug-my-permissions.
archive-helper	/archive-channel [limit] [before-ts]; Archive Thread message shortcut. Cursor-based history with Markdown, timestamps, edit markers, attachment links and source links.
moderation	/mod-summary, /mod-guidelines, message/channel/member/group commands and conditional workspace removal described below.
ui-showcase	/ui-showcase, /demo-message, /demo-form, /demo-search, /find, /demo-review, /demo-confirm, /demo-pager, /demo-cards, /browse, /demo-selects, /demo-alias, /demo-alias-alt.

Commands that open forms require a real trigger ID when running on Slack. Autocomplete is presented through an external select inside a modal; Slack does not support Discord-style slash-option autocomplete. For search commands, omit text to open the suggestion form, or supply text for direct results.

Configure persistent and authorized bots

Pass a JSON file with `--bot-config-file`. Only declared fields are accepted into `ctx.config`; host credentials never come from that file. Paths are relative to the process working directory unless absolute. Create parent directories before running a bot, because the SQLite module does not create directories.

For Knowledge Base:

```
{
  "dbPath": "/tmp/slack-knowledge-demo.sqlite",
  "captureEnabled": true,
  "captureThreshold": 0.65,
  "captureChannels": "C_CHANNEL",
  "reviewLimit": 5,
  "seedEntries": true,
  "reactionPromoteEmojis": "brain,pushpin",
  "trustedReviewerIds": "U_REVIEWER",
  "trustedReviewerGroups": ""
}
```

Knowledge Base allows all reviewers when both reviewer lists are empty, matching the source example's default. Set the lists to restrict review mutations and reaction promotion. Reviewer groups use Slack user-group membership, not roles. Manual teaching remains available to other users. Persistent records are scoped by workspace, and review changes record actor and action in the local audit table.

Custom KB declares dbPath and searchLimit. Its link uniqueness is scoped by workspace and URL. Saving the same URL updates the existing entry. The default files are ./slack-custom-kb.sqlite and ./slack-knowledge.sqlite respectively.

Show Space requires explicit management authorization:

```
{
  "dbPath": "/tmp/slack-shows-demo.sqlite",
  "managerIds": "U_MANAGER",
  "managerGroups": "",
  "announcementChannel": "C_ANNOUNCEMENTS",
  "seedShows": false
}
```

Both management lists empty means denied. Show dates use the source example's local-date parser. Announcement publication records the returned channel and message timestamp before pinning; pin failure does not lose that reference. Cancel/archive updates that stored message and removes its pin. The bundled seed shows are examples from May 2026 and therefore normally appear as past shows.

Moderation boundaries

Moderation requires moderatorIds or moderatorGroups. The available operations include message listing/fetching, pin/unpin/list-pins, bot-authored message deletion, channel info/topic, workspace info, user lookup/listing, group lookup and adding members, and channel removal. User-group updates read the current members and send the complete resulting list; Slack's API replaces membership rather than appending one member. Concurrent external group changes can race this operation.

Set useUserToken: true only when deliberately using an imported user token for message deletion and group membership changes. The explicit deleteAsUser and setMembersAsUser capabilities use that token; they never silently fall back to another identity. Slack still enforces the user's permissions and granted scopes. Without that setting these operations use the bot token.

/mod-remove-workspace-user USER_ID additionally requires enableWorkspaceRemoval: true and a separately imported Enterprise user token with admin.users:write. It is distinct from /mod-kick-channel USER_ID. The normal developer installation produces bot and

Socket Mode tokens, not this administrative user authorization. There is no live admin test in this project.

Discord timeouts, bans/unbans, channel slowmode and role permission hierarchies have no direct same-semantics implementation here. Slack thread membership is also different: threads are replies to messages, not independently joined channels. The Support bot names channel join/leave operations explicitly.

Offline validation and live operation

```
go run ./cmd/slack-bot bots simulate ui-showcase \
  --event-file examples/slack-bots/fixtures/ui-view.json
go run ./cmd/slack-bot bots invoke unified-demo status
go test ./pkg/slackcli ./internal/jsslack ./internal/slacktransport
```

Simulation records side effects. Its default recorder returns empty data for read APIs; use injected service fixtures in Go tests for workflows that need realistic users, histories or groups. Each simulation starts a new in-memory store. SQLite state can persist if the supplied configuration points to the same file across invocations.

The workflow tests cover database reopen/isolation, Poker state, reviewer and manager denial, exact pin references, multi-page archives, UI state and modal ACKs. The registration acceptance test checks source-derived mappings across all thirteen bot names. These checks do not prove every Slack plan, permission combination or API response. New ports have not been live-installed as part of this implementation; the original Ping/UI Showcase connection was the earlier live-tested baseline.

```
go run ./cmd/slack-bot bots run knowledge-base --profile my-kb \
  --bot-config-file /tmp/knowledge-config.json --log-level debug
```

Use tmux for the running process. For a large archive, raise `--timeout-ms` within the CLI's 60,000 ms maximum and reduce the requested message limit if needed. An API rate limit fails the operation rather than silently exporting partial history. Generated files are limited to 8 MiB.

Implementation references

- `examples/slack-bots/`: native scripts and shared domain helpers.
- `internal/jsslack/`: registration, UI builders, dispatch, database ownership and local verbs.
- `pkg/slackbot/operations.go`: named operation allowlist.
- `internal/slacktransport/operations.go`: Web API and external file transfer.
- `pkg/slackcli/port_inventory_test.go`: source-derived registration coverage.
- `pkg/slackcli/port_workflows_test.go`: multi-step behavioral evidence.
- `ttp/2026/09/15/SLACK-PORT-001--port-discord-example-bots-to-native-slack-workflows/`: intern guide, complete mapping, archived sources and chronological diary.